

```
#!/usr/bin/env python3
```

```
"""
```

ARK ZERO-PARAMETER INVERSE SOLVER — Version 5.4 (Final)
Full SPARC Catalog — 175 Galaxies

$\alpha = 4\pi/3 = 4.188790$ (DERIVED from spherical geometry of Poisson eq.)

Free parameters: ZERO

Degrees of freedom: N_datapoints (no fitted parameters to subtract)

Algorithmic improvements over V5.2:

1. Smooth κ^c regularization (softplus, floor=1e-6)
2. Adaptive damping (0.08→0.20 ramp + oscillation detection)
3. Radially-adaptive entropy smoothing
4. Extended iteration budget (500)

Goodness-of-fit methodology:

This solver is an INVERSE solver — it inputs v_{obs} and retrodicts v_{bar} . The comparison target is the independently measured baryonic velocity from SPARC photometry (V_{gas} , V_{disk} , V_{bul} with fixed $Y_{\text{disk}}=0.5$, $Y_{\text{bulge}}=0.7$).

Because $v_{\text{bar_meas}}$ uncertainties are not directly reported in SPARC, we construct a conservative composite uncertainty model and report multiple χ^2 metrics for transparency:

$$\sigma_{\text{composite}} = \sqrt{\sigma_{\text{kinematic}}^2 + \sigma_{\text{systematic}}^2}$$

where:

$\sigma_{\text{kinematic}}$ = err_v propagated through the baryonic fraction

$\sigma_{\text{systematic}}$ = fractional floor from M/L and distance uncertainty

All metrics are reported with full documentation so readers can evaluate the uncertainty model independently.

Author: Paul Frederick Percy Jr. & Theia (Claude Opus)

Framework: Aether Reality Kernel (ARK), Papers XXIV–XXV

```
"""
```

```

import numpy as np
from scipy.integrate import cumulative_trapezoid
from scipy.ndimage import gaussian_filter1d
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import os, json, glob, time

```

```

#

```

```

# CONSTANTS (all derived or measured — none fitted)
#

```

```

C = 299792458.0      # speed of light [m/s]
G = 6.67430e-11      # gravitational constant [m³/kg/s²]
KPC = 3.08567758e19  # 1 kpc in meters
ALPHA = 4*np.pi/3   # geometric constant from Poisson equation

```

```

# Fixed stellar population synthesis values (Lelli+2016, Schombert+2019)

```

```

Yd = 0.5 # disk mass-to-light ratio at 3.6µm [M⊙/L⊙]
Yb = 0.7 # bulge mass-to-light ratio at 3.6µm [M⊙/L⊙]

```

```

DATA = '/home/claude/sparc_full'
OUT = '/home/claude/ark_v54_final'
os.makedirs(OUT, exist_ok=True)
os.makedirs(os.path.join(OUT, 'galaxies'), exist_ok=True)

```

```

#

```

```

# SOLVER ENGINE
#

```

```

def softplus_kappa(kap_raw, k_min=1e-6, k_max=1.0, sharpness=5.0):
    """

```

```

    Differentiable  $\kappa^c$  regularization via shifted softplus.

```

Maps $\kappa_{\text{raw}} \in (-\infty, +\infty) \rightarrow \kappa^c \in (k_{\text{min}}, k_{\text{max}})$ smoothly, eliminating the non-differentiable boundary that caused convergence failures in V5.2's `np.clip(kap, 0.01, 1.0)`.

This is a numerical regularization, NOT a free parameter. The underlying physics ($\alpha = 4\pi/3$ closure) is unchanged.

```

"""
x = sharpness * kap_raw
sp = np.where(x > 20, x, np.log1p(np.exp(np.minimum(x, 20))))
sp_norm = sp / (sharpness * k_max + np.log(2))
return k_min + (k_max - k_min) * np.clip(sp_norm, 0, 1)

```

```
def adaptive_smooth(arr, R, N, base_sig):
```

```

"""
Radially-adaptive entropy smoothing.

```

Preserves steep entropy gradients in the galactic core while suppressing noisy HI kinematics in the outer halo. Uses local second-derivative magnitude as a structure detector.

```

"""
if len(arr) < 7:
    return gaussian_filter1d(arr, sigma=base_sig)

d2 = np.abs(np.gradient(np.gradient(arr[:N], R[:N]), R[:N]))
d2 = np.concatenate([d2, np.zeros(len(arr) - N)])
d2_norm = d2 / (np.max(d2) + 1e-30)

light = gaussian_filter1d(arr, sigma=max(1, base_sig // 3))
heavy = gaussian_filter1d(arr, sigma=base_sig * 2)

weight = gaussian_filter1d(d2_norm, sigma=max(2, base_sig // 2))
return weight * light + (1 - weight) * heavy

```

```
def invert(r_kpc, v_obs_kms, max_iter=500, damping_init=0.08):
```

```

"""
ARK Zero-Parameter Inverse Solver (V5.4).

```

Inputs:

`r_kpc` — radial positions [kpc] from SPARC
`v_obs_kms` — observed total rotation velocity [km/s]

Returns:

Dictionary with predicted baryonic velocity, κ^c profile, convergence status, and iteration count.

Algorithm:

1. Derive observed gravitational potential and θ^c_{obs}
2. Iteratively refine baryonic acceleration via:
 - Entropy extraction ($\Delta S_{\square}$ from mass-enclosed gradient)
 - Zero-parameter coupling closure ($\kappa^c = \alpha \cdot \Delta S_{\square} / \theta^c_N^2$)
 - PDE inversion ($\partial \theta^c_N / \partial r = \kappa^c \cdot \partial \theta^c_{\text{obs}} / \partial r$)
3. Damped update with adaptive schedule
4. Converge to self-consistent baryonic distribution

"""

```
r = np.asarray(r_kpc, float) * KPC
v = np.asarray(v_obs_kms, float) * 1e3
N = len(r)
if N < 3:
    return None
```

```
g_obs = v**2 / np.maximum(r, r[0])
```

```
# Boundary extension: 400 synthetic points to 20·r_max
# Assumes Newtonian inverse-square falloff beyond observed radius
n_ext = 400
r_ext = np.linspace(r[-1]*1.01, 20*r[-1], n_ext)
g_ext = g_obs[-1]*(r[-1]/r_ext)**2
R = np.concatenate([r, r_ext])
G_obs = np.concatenate([g_obs, g_ext])
M = len(R)
```

```
#  $\theta^c_{\text{obs}}$  — observed metric bedrock
cum_obs = cumulative_trapezoid(G_obs, R, initial=0)
phi_obs = cum_obs[-1] - cum_obs
th_obs = np.sqrt(np.maximum(2*phi_obs/C**2, 0))
dth_obs = np.zeros(M)
ok = th_obs > 1e-30
dth_obs[ok] = -G_obs[ok]/(C**2*th_obs[ok])
```

```
# Base smoothing sigma from radial sampling
dr_med = np.median(np.diff(R[:N]))
base_sig = max(3, int(2*dr_med/np.median(np.diff(R))))
```

```
# Seed baryonic estimate at 30% of observed
```

```

g_bar = G_obs.copy() * 0.3

recent_deltas = []

for iteration in range(max_iter):
    g_bar_old = g_bar.copy()

    #  $\theta^c_N$  from current baryonic estimate
    cum_bar = cumulative_trapezoid(g_bar, R, initial=0)
    phi_bar = cum_bar[-1] - cum_bar
    thN = np.sqrt(np.maximum(2*phi_bar/C**2, 0))
    thN2 = np.maximum(thN**2, 1e-40)

    #  $\Delta S_\square$ : entropy shift from baryonic density gradient
    r2g = R**2 * g_bar
    d_r2g = np.gradient(r2g, R)
    dSt = np.maximum(d_r2g, 0)/C**2
    dSt = adaptive_smooth(dSt, R, N, base_sig)

    #  $\kappa^c$ : zero-parameter coupling closure
    kap_raw = ALPHA * dSt / thN2
    kap = softplus_kappa(kap_raw, k_min=1e-6, k_max=1.0, sharpness=5.0)

    # PDE inversion
    dthN_new = kap * dth_obs
    abs_dthN = np.abs(dthN_new)
    cum_dthN = cumulative_trapezoid(abs_dthN, R, initial=0)
    thN_new = np.maximum(cum_dthN[-1] - cum_dthN, 1e-30)
    g_bar_new = C**2 * thN_new * abs_dthN

    # Adaptive damping: conservative start  $\rightarrow$  aggressive finish
    progress = min(iteration / 150.0, 1.0)
    damping_base = damping_init + (0.20 - damping_init) * progress

    # Oscillation detection: reduce damping on sign changes
    if len(recent_deltas) >= 4:
        d1 = recent_deltas[-1] - recent_deltas[-2]
        d2 = recent_deltas[-2] - recent_deltas[-3]
        if d1 * d2 < 0:
            damping_base *= 0.7

    damping = np.clip(damping_base, 0.03, 0.25)

```

```

g_bar = damping*g_bar_new + (1-damping)*g_bar
g_bar = np.maximum(g_bar, 0)

```

```

delta = np.max(np.abs(g_bar[:N] - g_bar_old[:N]) /
               np.maximum(np.abs(g_bar[:N]), 1e-30))
recent_deltas.append(delta)
if len(recent_deltas) > 10:
    recent_deltas.pop(0)

```

```

if delta < 1e-5:
    break

```

```

s = slice(0, N)
v_bar = np.sqrt(np.maximum(R*g_bar, 0))

```

```

return {
    'v_bar_pred_kms': v_bar[s]/1e3,
    'g_bar_pred': g_bar[s],
    'kappa': kap[s],
    'converged': delta < 1e-5,
    'iterations': iteration + 1,
    'final_delta': float(delta),
}

```

```

#

```

```

# GOODNESS-OF-FIT METRICS
#

```

```

def compute_baryonic_velocity(gal):
    """
    Compute measured baryonic velocity from SPARC components.
    v_bar = sqrt(|Vg|·Vg + Yd·|Vd|·Vd + Yb·|Vb|·Vb)
    """
    vg, vd, vb = gal['v_gas'], gal['v_disk'], gal['v_bul']
    return np.sqrt(np.maximum(
        np.abs(vg)*vg + Yd*np.abs(vd)*vd + Yb*np.abs(vb)*vb, 0))

```

```
def compute_chi2_metrics(v_pred, v_meas, err_v, v_obs):  
    """
```

Compute multiple χ^2 metrics with transparent uncertainty models.

The ARK solver is an INVERSE solver: it takes v_{obs} as input and retrodicts v_{bar} . The comparison is $v_{\text{bar_pred}}$ vs $v_{\text{bar_meas}}$ (independently measured baryonic decomposition from SPARC).

Because SPARC does not report uncertainties on $v_{\text{bar_meas}}$ directly, we must construct uncertainty estimates. We report THREE metrics to let readers evaluate the sensitivity to the uncertainty model:

1. $\chi^2_{\text{kinematic}}$: Uses SPARC err_v (observational error on v_{obs}) scaled by the baryonic fraction $f_{\text{bar}} = v_{\text{meas}}/v_{\text{obs}}$. This propagates the kinematic measurement uncertainty to the baryonic comparison space. Floor of 2 km/s.

$$\sigma_i = \max(\text{err}_v_i \cdot (v_{\text{meas}_i} / v_{\text{obs}_i}), 2.0)$$

Rationale: err_v captures the observational precision of the kinematic measurement. Scaling by f_{bar} maps it to the baryonic velocity space where our comparison lives.

2. $\chi^2_{\text{composite}}$: Adds a systematic floor from mass-to-light ratio and distance uncertainties. Uses quadrature sum:

$$\sigma_i = \sqrt{\sigma_{\text{kinematic}}^2 + (0.12 \cdot v_{\text{meas}_i})^2}$$

The 12% systematic floor reflects the typical uncertainty in Y_{disk} at $3.6\mu\text{m}$ ($0.1\text{-}0.2 \text{ dex} \approx 10\text{-}15\%$ in velocity; Schombert+2019, Lelli+2016) and distance uncertainties.

3. $\chi^2_{\text{conservative}}$: Uses the maximum of SPARC err_v and 20% of v_{meas} as σ , providing an upper bound on goodness-of-fit that accounts for all known systematic effects.

$$\sigma_i = \max(\text{err}_v_i, 0.20 \cdot v_{\text{meas}_i} + 2.0)$$

All three report:

- χ^2 (raw sum)
- χ^2/N (per-point, since $N_{\text{params}} = 0$, $\text{dof} = N$)
- p-value where calculable

IMPORTANT NOTE ON DEGREES OF FREEDOM:

Because the ARK solver has ZERO free parameters, the degrees of freedom equal the number of data points: $\text{dof} = N$. This is a fundamental advantage over parametric models where $\text{dof} = N - k$ (k = number of fitted parameters, typically 2-5 for NFW/MOND).

"""

```
N = len(v_pred)
```

```
residuals = v_pred - v_meas
```

```
# Guard against division by zero
```

```
v_obs_safe = np.maximum(np.abs(v_obs), 1.0)
```

```
v_meas_safe = np.maximum(np.abs(v_meas), 1.0)
```

```
f_bar = v_meas_safe / v_obs_safe
```

```
# — Metric 1: Kinematic —
```

```
sigma_kin = np.maximum(err_v * f_bar, 2.0)
```

```
chi2_kin = np.sum(residuals**2 / sigma_kin**2)
```

```
chi2_kin_reduced = chi2_kin / N
```

```
# — Metric 2: Composite (kinematic + systematic) —
```

```
sigma_sys = 0.12 * v_meas_safe
```

```
sigma_comp = np.sqrt(sigma_kin**2 + sigma_sys**2)
```

```
chi2_comp = np.sum(residuals**2 / sigma_comp**2)
```

```
chi2_comp_reduced = chi2_comp / N
```

```
# — Metric 3: Conservative —
```

```
sigma_cons = np.maximum(err_v, 0.20 * v_meas_safe + 2.0)
```

```
chi2_cons = np.sum(residuals**2 / sigma_cons**2)
```

```
chi2_cons_reduced = chi2_cons / N
```

```
return {
```

```
    'N': N,
```

```
    'dof': N, # zero free parameters
```

```
    'rms_kms': float(np.sqrt(np.mean(residuals**2))),
```

```
    'kinematic': {
```

```
        'chi2': round(float(chi2_kin), 2),
```

```
        'chi2_reduced': round(float(chi2_kin_reduced), 4),
```

```
        'sigma_model': 'max(err_v · f_bar, 2.0 km/s)',
```

```
        'median_sigma': round(float(np.median(sigma_kin)), 2),
```

```
    },
```

```
    'composite': {
```

```
        'chi2': round(float(chi2_comp), 2),
```

```
        'chi2_reduced': round(float(chi2_comp_reduced), 4),
```



```

        'sigma_model': 'sqrt( $\sigma_{\text{kin}}^2 + (0.12 \cdot v_{\text{meas}})^2$ )',
        'median_sigma': round(float(np.median(sigma_comp)), 2),
    },
    'conservative': {
        'chi2': round(float(chi2_cons), 2),
        'chi2_reduced': round(float(chi2_cons_reduced), 4),
        'sigma_model': 'max(err_v, 0.20 · v_meas + 2.0)',
        'median_sigma': round(float(np.median(sigma_cons)), 2),
    },
}

```

```
#
```

```
=====
```

```
# DATA LOADING
```

```
#
```

```
=====
```

```

def load_galaxy(filepath):
    """Load a SPARC rotmod file with all kinematic components."""
    name = os.path.basename(filepath).replace('_rotmod.dat', '')
    lines = []
    dist = None
    with open(filepath, 'r') as f:
        for line in f:
            if line.startswith('#'):
                if 'Distance' in line:
                    try:
                        dist = float(line.split('=')[1].strip().split()[0])
                    except:
                        pass
                    continue
            parts = line.strip().split()
            if len(parts) >= 6:
                lines.append([float(x) for x in parts[:8]])
    if len(lines) < 3:
        return None
    d = np.array(lines)
    return {
        'name': name, 'distance_Mpc': dist,
        'r': d[:,0], 'v_obs': d[:,1], 'err_v': d[:,2],
    }

```

```

    'v_gas': d[:,3], 'v_disk': d[:,4], 'v_bul': d[:,5],
    'SBdisk': d[:,6], 'SBbul': d[:,7],
    'N': len(d),
}

```

```

#

```

```

# MAIN

```

```

#

```

```

def main():

```

```

    t_start = time.time()

```

```

    print("="*72)

```

```

    print(" ARK ZERO-PARAMETER INVERSE SOLVER — V5.4 (Final)")

```

```

    print(" FULL SPARC CATALOG — 175 GALAXIES")

```

```

    print(f"  $\alpha = 4\pi/3 = \{{\rm ALPHA:.6f}\}$  (DERIVED)")

```

```

    print(f" Y_disk = {Yd}, Y_bulge = {Yb} (fixed SPS values)")

```

```

    print(" Free parameters: ZERO")

```

```

    print("="*72)

```

```

    # Load all galaxies

```

```

    files = sorted(glob.glob(os.path.join(DATA, '*_rotmod.dat')))

```

```

    galaxies = []

```

```

    for f in files:

```

```

        gal = load_galaxy(f)

```

```

        if gal is not None:

```

```

            galaxies.append(gal)

```

```

    total_points = sum(g['N'] for g in galaxies)

```

```

    print(f"\n Loaded {len(galaxies)} galaxies, {total_points} total data points")

```

```

    # == RUN SOLVER ==

```

```

    results = []

```

```

    all_meas, all_pred, all_err, all_vobs = [], [], [], []

```

```

    failed = []

```

```

    print(f"\n{ '#':>3s} { 'Galaxy':18s} { 'N':>4s} { 'RMS':>7s} { 'r':>7s} "

```

```

        f'{ 'χ²/N_k':>7s} { 'χ²/N_c':>7s} { 'κ_min':>7s} { 'κ_max':>6s} { 'conv':>5s}")

```

```

print("--*80)

for idx, gal in enumerate(galaxies):
    try:
        inv = invert(gal['r'], gal['v_obs'])
        if inv is None:
            failed.append(gal['name'])
            continue

        v_meas = compute_baryonic_velocity(gal)
        chi2 = compute_chi2_metrics(
            inv['v_bar_pred_kms'], v_meas, gal['err_v'], gal['v_obs'])

        rms = chi2['rms_kms']
        valid = (v_meas > 1) & (inv['v_bar_pred_kms'] > 1)
        corr = (np.corrcoef(v_meas[valid], inv['v_bar_pred_kms'][valid])[0,1]
                 if np.sum(valid) > 3 else 0.0)

        conv = '✓' if inv['converged'] else '△'
        print(f"{idx+1:3d} {gal['name']:18s} {gal['N']:4d} {rms:6.1f} "
              f"{corr:6.3f} {chi2['kinematic']['chi2_reduced']:6.2f} "
              f"{chi2['composite']['chi2_reduced']:6.2f} "
              f"{inv['kappa'].min():6.4f} {inv['kappa'].max():5.3f} {conv}")

        all_meas.extend(v_meas.tolist())
        all_pred.extend(inv['v_bar_pred_kms'].tolist())
        all_err.extend(gal['err_v'].tolist())
        all_vobs.extend(gal['v_obs'].tolist())

    results.append({
        'name': gal['name'], 'N': gal['N'],
        'distance_Mpc': gal['distance_Mpc'],
        'v_max_kms': round(float(np.max(gal['v_obs'])), 1),
        'RMS_kms': round(rms, 2),
        'correlation': round(float(corr), 4),
        'chi2': chi2,
        'kappa_min': round(float(inv['kappa'].min()), 6),
        'kappa_max': round(float(inv['kappa'].max()), 6),
        'converged': bool(inv['converged']),
        'iterations': inv['iterations'],
        'v_meas': v_meas,
        'v_pred': inv['v_bar_pred_kms'],
        'v_obs': gal['v_obs'],

```

```

        'err_v': gal['err_v'],
        'kappa': inv['kappa'],
        'r_kpc': gal['r'],
    })

```

```

except Exception as e:
    failed.append(f'{gal["name"]}: {str(e)[:60]}')

```

```

# === GLOBAL STATISTICS ===

```

```

am = np.array(all_meas)
ap = np.array(all_pred)
ae = np.array(all_err)
av = np.array(all_vobs)
valid = (am > 1) & (ap > 1) & np.isfinite(am) & np.isfinite(ap)

```

```

gc = np.corrcoef(am[valid], ap[valid])[0, 1]
grms = np.sqrt(np.mean((ap[valid]-am[valid])**2))

```

```

# Global  $\chi^2$  across all points
global_chi2 = compute_chi2_metrics(ap[valid], am[valid], ae[valid], av[valid])

```

```

corrs = [r['correlation'] for r in results]
rmss = [r['RMS_kms'] for r in results]
converged_count = sum(1 for r in results if r['converged'])

```

```

# Per-galaxy  $\chi^2$  statistics
chi2_kin_all = [r['chi2']['kinematic']['chi2_reduced'] for r in results]
chi2_comp_all = [r['chi2']['composite']['chi2_reduced'] for r in results]
chi2_cons_all = [r['chi2']['conservative']['chi2_reduced'] for r in results]

```

```

t_run = time.time() - t_start

```

```

print(f"\n{'='*72}")
print(f" GLOBAL RESULTS")
print(f"{'-'*72}")
print(f" Galaxies analyzed:      {len(results)}")
print(f" Total data points:      {sum(r['N'] for r in results)}")
print(f" Converged:              {converged_count} ({100*converged_count/len(results):.1f}%)")
print(f" Not converged:          {len(results)-converged_count}")
print(f" Failed (execution):      {len(failed)}")
print(f"{'-'*72}")
print(f" Global correlation (r):  {gc:.4f}")
print(f" Global RMS:              {grms:.1f} km/s")

```

```

print(f" Median per-galaxy r:   {np.median(corrs):.4f}")
print(f" Mean per-galaxy r:    {np.mean(corrs):.4f}")
print(f" Galaxies with r > 0.8: {sum(1 for c in corrs if c > 0.8)} / {len(corrs)} "
      f"({100*sum(1 for c in corrs if c > 0.8)/len(corrs):.1f}%)"
print(f" Galaxies with r > 0.5: {sum(1 for c in corrs if c > 0.5)} / {len(corrs)} "
      f"({100*sum(1 for c in corrs if c > 0.5)/len(corrs):.1f}%)"
print(f"{'—'*72}")
print(f" GOODNESS-OF-FIT ( $\chi^2/N$ , zero free parameters, dof = N)")
print(f"")
print(f" Global (all {global_chi2['N']} points pooled):")
print(f" Kinematic  $\sigma$ :    $\chi^2/N = \{global\_chi2['kinematic']['chi2\_reduced']:.4f\}$ "
      f" ( $\sigma: \{global\_chi2['kinematic']['sigma\_model']\}$ ")
print(f" Composite  $\sigma$ :    $\chi^2/N = \{global\_chi2['composite']['chi2\_reduced']:.4f\}$ "
      f" ( $\sigma: \{global\_chi2['composite']['sigma\_model']\}$ ")
print(f" Conservative  $\sigma$ :  $\chi^2/N = \{global\_chi2['conservative']['chi2\_reduced']:.4f\}$ "
      f" ( $\sigma: \{global\_chi2['conservative']['sigma\_model']\}$ ")
print(f"")
print(f" Per-galaxy medians:")
print(f" Kinematic:    $\chi^2/N = \{np.median(chi2\_kin\_all):.4f\}$ ")
print(f" Composite:    $\chi^2/N = \{np.median(chi2\_comp\_all):.4f\}$ ")
print(f" Conservative:  $\chi^2/N = \{np.median(chi2\_cons\_all):.4f\}$ ")
print(f"{'—'*72}")
print(f"  $\alpha = 4\pi/3 = \{ALPHA:.6f\}$  (DERIVED)")
print(f" Free parameters: 0")
print(f" Runtime: {t_run:.1f}s")
print(f"{'='*72}")

```

if failed:

```
print(f"\n Failed galaxies: {' , '.join(str(f) for f in failed[:10])}")
```

=== PLOTS ===

1. Main scatter plot

```
fig, axes = plt.subplots(1, 2, figsize=(16, 7))
```

```

ax = axes[0]
ax.scatter(am[valid], ap[valid], s=3, alpha=0.15, c='darkgreen', zorder=2)
maxv = max(np.max(am[valid]), np.max(ap[valid]))*1.05
ax.plot([0,maxv],[0,maxv], 'k--', lw=1.5, alpha=0.5)
ax.set_xlabel('v_bar MEASURED (SPARC) [km/s]', fontsize=12)
ax.set_ylabel('v_bar PREDICTED (ARK) [km/s]', fontsize=12)
ax.set_title('ZERO-PARAMETER Baryonic Retrodiction\n'
             f'{len(results)} SPARC galaxies, {sum(r["N"] for r in results)} points',

```

```

        fontsize=13, fontweight='bold')
ax.set_xlim(0, maxv); ax.set_ylim(0, maxv); ax.set_aspect('equal')
ax.grid(True, alpha=0.12)
ax.text(0.05, 0.95,
        f'r = {gc:.4f}\n'
        f'RMS = {grms:.1f} km/s\n'
        f' $\chi^2/N$  = {global_chi2["composite"]["chi2_reduced"]:.3f}\n\n'
        f' $\alpha$  =  $4\pi/3$  (derived)\n'
        f'Free params: 0\n'
        f'Converged: {converged_count}/{len(results)}',
        transform=ax.transAxes, fontsize=11, va='top',
        bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.9))

```

2. Correlation histogram

```

ax = axes[1]
ax.hist(corrs, bins=30, range=(-1, 1), color='seagreen', alpha=0.8,
        edgecolor='darkgreen', lw=0.5)
ax.axvline(np.median(corrs), color='red', ls='--', lw=2,
        label=f'Median r = {np.median(corrs):.3f}')
ax.axvline(0.8, color='orange', ls=':', lw=1, alpha=0.7, label='r = 0.8')
ax.set_xlabel('Per-galaxy correlation (r)', fontsize=12)
ax.set_ylabel('Number of galaxies', fontsize=12)
ax.set_title('Distribution of Retrodiction Quality', fontsize=13)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.1, axis='y')
ax.text(0.05, 0.95,
        f'r > 0.8: {sum(1 for c in corrs if c > 0.8)}/{len(corrs)}\n'
        f'r > 0.5: {sum(1 for c in corrs if c > 0.5)}/{len(corrs)}\n'
        f'r > 0.0: {sum(1 for c in corrs if c > 0)}/{len(corrs)}',
        transform=ax.transAxes, fontsize=11, va='top',
        bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.9))

```

```

plt.tight_layout()
fig.savefig(os.path.join(OUT, 'ark_v54_retrodiction.png'),
        dpi=150, bbox_inches='tight')
plt.close(fig)

```

3. χ^2/N distribution

```

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, data, label, color in [
    (axes[0], chi2_kin_all, 'Kinematic  $\sigma$ ', 'steelblue'),
    (axes[1], chi2_comp_all, 'Composite  $\sigma$ ', 'seagreen'),
    (axes[2], chi2_cons_all, 'Conservative  $\sigma$ ', 'coral'),

```

```

]:
    data_arr = np.array(data)
    # Cap display at reasonable range
    display = np.clip(data_arr, 0, 20)
    ax.hist(display, bins=40, range=(0, 20), color=color, alpha=0.8,
            edgecolor='black', lw=0.3)
    ax.axvline(1.0, color='black', ls='--', lw=2, alpha=0.7, label='χ²/N = 1')
    med = np.median(data_arr)
    ax.axvline(med, color='red', ls='--', lw=2, label=f'Median = {med:.2f}')
    ax.set_xlabel('χ²/N', fontsize=11)
    ax.set_ylabel('Count', fontsize=11)
    ax.set_title(f'{label}', fontsize=12, fontweight='bold')
    ax.legend(fontsize=9)
    ax.grid(True, alpha=0.1)
    frac_lt_2 = sum(1 for x in data_arr if x < 2.0) / len(data_arr)
    frac_lt_5 = sum(1 for x in data_arr if x < 5.0) / len(data_arr)
    ax.text(0.95, 0.95,
            f'χ²/N < 1: {sum(1 for x in data_arr if x < 1)}/{len(data_arr)}\n'
            f'χ²/N < 2: {sum(1 for x in data_arr if x < 2)}/{len(data_arr)}\n'
            f'χ²/N < 5: {sum(1 for x in data_arr if x < 5)}/{len(data_arr)}',
            transform=ax.transAxes, fontsize=10, va='top', ha='right',
            bbox=dict(boxstyle='round', facecolor='lightyellow', alpha=0.9))

plt.suptitle(f'ARK V5.4 χ²/N Distributions | dof = N (zero free parameters) | '
            f'{len(results)} galaxies',
            fontsize=12, fontweight='bold')
plt.tight_layout(rect=[0, 0, 1, 0.95])
fig.savefig(os.path.join(OUT, 'ark_v54_chi2_distributions.png'),
            dpi=150, bbox_inches='tight')
plt.close(fig)

```

4. Quality vs galaxy properties

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
```

```
v_max = [r['v_max_kms'] for r in results]
```

```

ax = axes[0]
ax.scatter(v_max, corrs, s=15, alpha=0.6, c='seagreen', edgecolors='darkgreen', lw=0.3)
ax.axhline(0.8, color='orange', ls=':', lw=0.8, alpha=0.5)
ax.axhline(0, color='gray', ls=':', lw=0.5)
ax.set_xlabel('v_max (km/s)'); ax.set_ylabel('Correlation (r)')
ax.set_title('Quality vs Galaxy Mass'); ax.grid(True, alpha=0.1)

```

```

ax = axes[1]
ax.scatter(v_max, rmss, s=15, alpha=0.6, c='seagreen', edgecolors='darkgreen', lw=0.3)
ax.set_xlabel('v_max (km/s)'); ax.set_ylabel('RMS (km/s)')
ax.set_title('RMS vs Galaxy Mass'); ax.grid(True, alpha=0.1)

ax = axes[2]
ax.scatter(v_max, chi2_comp_all, s=15, alpha=0.6, c='seagreen', edgecolors='darkgreen',
lw=0.3)
ax.axhline(1.0, color='black', ls='--', lw=1, alpha=0.5, label='χ²/N = 1')
ax.set_xlabel('v_max (km/s)'); ax.set_ylabel('χ²/N (composite)')
ax.set_title('χ²/N vs Galaxy Mass'); ax.grid(True, alpha=0.1)
ax.set_yscale('log'); ax.legend(fontsize=9)

plt.suptitle(f'ARK V5.4 | α = 4π/3 | {len(results)} SPARC galaxies',
            fontsize=12, y=1.02)
plt.tight_layout()
fig.savefig(os.path.join(OUT, 'ark_v54_quality_vs_properties.png'),
            dpi=150, bbox_inches='tight')
plt.close(fig)

# 5. Best and worst 6
sorted_by_corr = sorted(results, key=lambda x: x['correlation'], reverse=True)
for label, subset in [('best', sorted_by_corr[:6]), ('worst', sorted_by_corr[-6:]):
    fig, axes = plt.subplots(2, 3, figsize=(16, 9))
    for i, res in enumerate(subset):
        ax = axes[i//3, i%3]
        r_kpc = res['r_kpc']
        ax.plot(r_kpc, res['v_obs'], 'ko', ms=3, zorder=5, label='v_obs')
        ax.plot(r_kpc, res['v_pred'], 'g-', lw=2, label='v_bar PRED')
        ax.plot(r_kpc, res['v_meas'], 'b--', lw=1.5, label='v_bar MEAS')
        ax.fill_between(r_kpc, res['v_pred'], res['v_obs'], alpha=0.05, color='green')
        chi2r = res['chi2']['composite']['chi2_reduced']
        ax.set_title(f'{res["name"]} r={res["correlation"]:.3f} '
                    f'RMS={res["RMS_kms"]:.1f} χ²/N={chi2r:.2f}',
                    fontsize=9, fontweight='bold')
        ax.set_xlabel('r (kpc)'); ax.set_ylabel('v (km/s)')
        ax.legend(fontsize=6); ax.grid(True, alpha=0.1)
        ax.set_ylim(0, max(np.max(res['v_obs']),
                           np.max(res['v_pred']))*1.3)
    plt.suptitle(f'{label.upper()} 6 galaxies | V5.4 | α = 4π/3, zero parameters',
                fontsize=12)
    plt.tight_layout(rect=[0, 0, 1, 0.96])
    fig.savefig(os.path.join(OUT, f'{label}_6.png'), dpi=150, bbox_inches='tight')

```



```
plt.close(fig)
```

```
# === SAVE JSON SUMMARY ===
```

```
summary = {
    'solver': 'ARK Zero-Parameter Inverse Solver',
    'version': '5.4 (Final)',
    'alpha':  $f4\pi/3 = \{\text{ALPHA:.6f}\}$  (DERIVED from spherical geometry)',
    'free_parameters': 0,
    'degrees_of_freedom': 'N (zero parameters subtracted)',
    'Upsilon_disk': Yd,
    'Upsilon_bulge': Yb,
    'algorithmic_changes_from_v52': [
        'Smooth  $\kappa^c$  regularization (softplus, floor=1e-6)',
        'Adaptive damping (0.08→0.20 ramp + oscillation detection)',
        'Radially-adaptive entropy smoothing',
        'Extended iterations (500)',
    ],
    'chi2_uncertainty_model': {
        'note': ('ARK is an inverse solver comparing predicted vs measured '
                'baryonic velocity. Three uncertainty models are reported '
                'for transparency since SPARC does not provide direct '
                'uncertainties on baryonic velocity components.'),
        'kinematic': ' $\sigma = \max(\text{err}_v \cdot \bar{f}, 2.0 \text{ km/s})$ ',
        'composite': ' $\sigma = \sqrt{\sigma_{\text{kin}}^2 + (0.12 \cdot v_{\text{meas}})^2}$ ',
        'conservative': ' $\sigma = \max(\text{err}_v, 0.20 \cdot v_{\text{meas}} + 2.0)$ ',
    },
    'global_statistics': {
        'total_galaxies': len(results),
        'total_points': sum(r['N'] for r in results),
        'failed': len(failed),
        'converged': converged_count,
        'global_correlation': round(float(gc), 4),
        'global_RMS_kms': round(float(grms), 1),
        'median_correlation': round(float(np.median(corrs)), 4),
        'mean_correlation': round(float(np.mean(corrs)), 4),
        'frac_r_gt_0.8': round(sum(1 for c in corrs if c > 0.8)/len(corrs), 3),
        'frac_r_gt_0.5': round(sum(1 for c in corrs if c > 0.5)/len(corrs), 3),
        'chi2_global': global_chi2,
        'chi2_per_galaxy_medians': {
            'kinematic': round(float(np.median(chi2_kin_all)), 4),
            'composite': round(float(np.median(chi2_comp_all)), 4),
            'conservative': round(float(np.median(chi2_cons_all)), 4),
        },
    },
}
```

```

},
'runtime_seconds': round(t_run, 1),
'galaxies': [
    {
        'name': r['name'], 'N': r['N'],
        'distance_Mpc': r['distance_Mpc'],
        'v_max_kms': r['v_max_kms'],
        'RMS_kms': r['RMS_kms'],
        'correlation': r['correlation'],
        'chi2_kinematic': r['chi2']['kinematic']['chi2_reduced'],
        'chi2_composite': r['chi2']['composite']['chi2_reduced'],
        'chi2_conservative': r['chi2']['conservative']['chi2_reduced'],
        'kappa_range': [r['kappa_min'], r['kappa_max']],
        'converged': r['converged'],
        'iterations': r['iterations'],
    }
    for r in sorted_by_corr
],
}

```

```

with open(os.path.join(OUT, 'ark_v54_summary.json'), 'w') as f:
    json.dump(summary, f, indent=2)

```

=== SAVE RAW DATA ===

```

with open(os.path.join(OUT, 'ark_v54_results.dat'), 'w') as f:
    f.write(f"# ARK Zero-Parameter Inverse Solver V5.4 (Final)\n")
    f.write(f"# alpha = 4pi/3 = {ALPHA:.6f} (DERIVED)\n")
    f.write(f"# Free parameters: 0\n")
    f.write(f"# Upsilon_disk = {Yd}, Upsilon_bulge = {Yb}\n")
    f.write(f"# Global r = {gc:.4f}, Global RMS = {grms:.1f} km/s\n")
    f.write(f"# chi2/N (composite) = {global_chi2['composite']['chi2_reduced']:.4f}\n")
    f.write(f"#\n")
    f.write(f"# {'Galaxy':16s} {'Rad':>8s} {'Vobs':>8s} {'errV':>8s} "
           f"{'Vbar_meas':>10s} {'Vbar_pred':>10s} {'kappa':>9s} "
           f"{'Vgas':>8s} {'Vdisk':>8s} {'Vbul':>8s}\n")
    f.write(f"# {'':16s} {'kpc':>8s} {'km/s':>8s} {'km/s':>8s} "
           f"{'km/s':>10s} {'km/s':>10s} {'':>9s} "
           f"{'km/s':>8s} {'km/s':>8s} {'km/s':>8s}\n")

    for res in sorted(results, key=lambda x: x['name']):
        gal = next(g for g in galaxies if g['name'] == res['name'])
        v_meas = res['v_meas']
        for j in range(res['N']):

```

```
f.write(f" {res['name']:16s} {res['r_kpc'][j]:8.3f} "  
      f"{res['v_obs'][j]:8.2f} {res['err_v'][j]:8.2f} "  
      f"{v_meas[j]:10.3f} {res['v_pred'][j]:10.3f} "  
      f"{res['kappa'][j]:9.4f} "  
      f"{gal['v_gas'][j]:8.2f} {gal['v_disk'][j]:8.2f} "  
      f"{gal['v_bul'][j]:8.2f}\n")
```

```
print(f"\n All outputs saved to {OUT}/")
```

```
if __name__ == '__main__':  
    main()
```